Agenda.

⇒ Google Typeahead Case Study.

⇒ 4 step process to approach HLD interviews.

#

1. MVP (Minimum Viable Product)
   - ↳ Product with minimum set of features.
2. Scale Estimation
   - ↳ Back of the Envelope calculations.
     - → No. of users | API requests.
     - → Amount of data.
       - ↳ Sharding is required or not.
     - → # of Read (Vs) Write queries.
       - ↳ # of Reads >>>> # of writes (Read Heavy)
       - ↳ # of writes >>>> # of Reads (Write Heavy)
       - ↳ # of Reads ≈ # of writes (R+W Heavy)
3. Design Tradeoffs | Design Choices
   - ↳ High Consistency (Vs) High Availability.
   - ↳ Latency

→ Can we afford data loss ?.

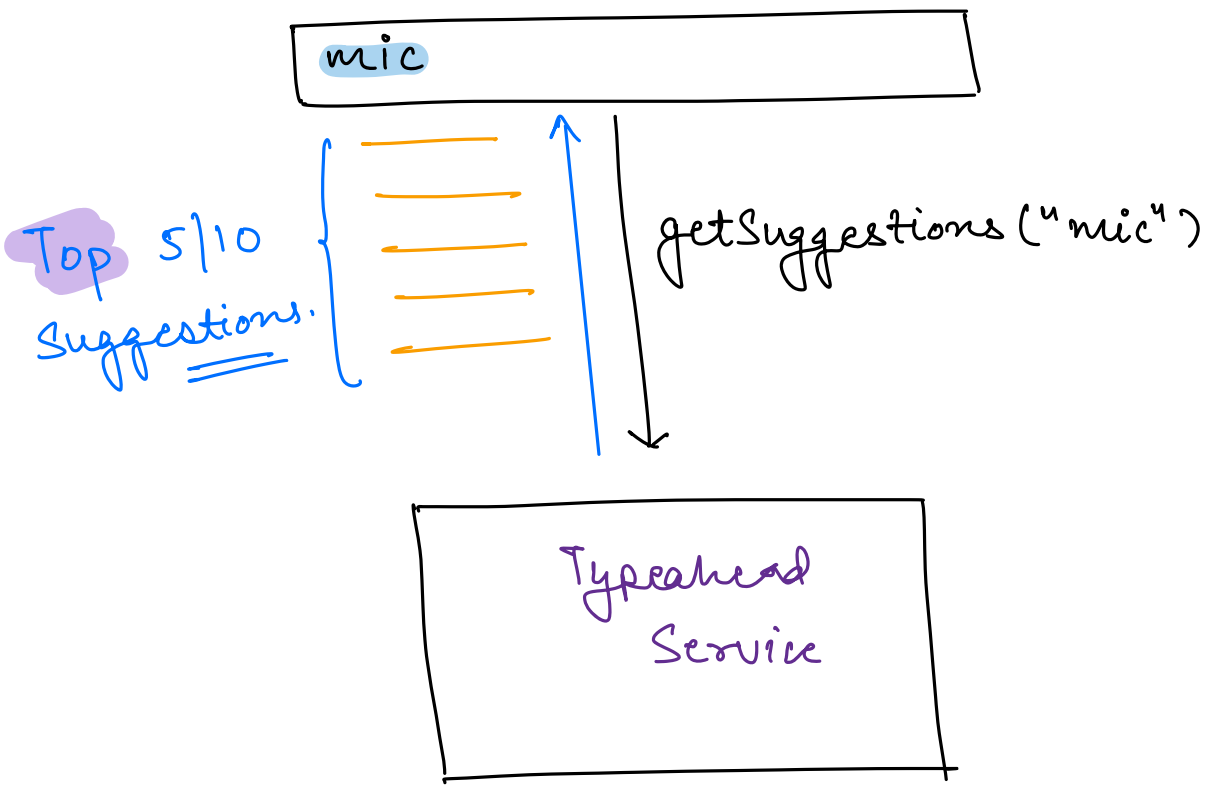
→ Stateful (vs) Stateless

→ SQL (vs) NOSQL.

4. Design Deepdive.

- → API's
- → System Diagram
- → Data flow.

Search Typeahead.



1. MVP.

→ Given the query prefix, give the top 5|10 Suggestion starting with the prefix.

→ Min. 3 characters.

→ 5 suggestions.

→ Relevant suggestion.

2. Scale Estimation.

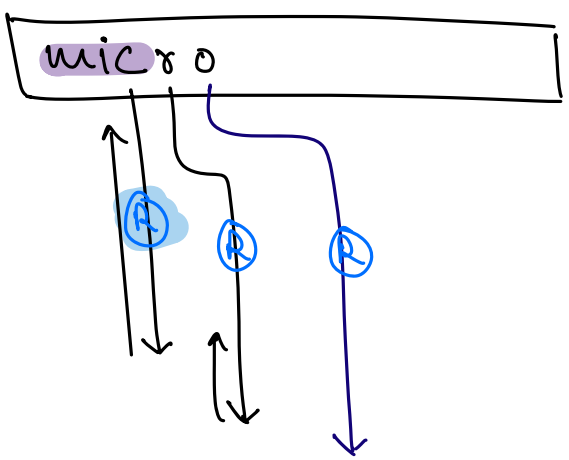
# of users = 3 B

DAU ~ 1.5 B.

Avg searches per day per user = 10

Avg searches per day = 10 x 1.5 B

= 15 B. searches | day.



Avg # of read calls to the Typeahead System per search = 5

Total read calls to the Typeahead = 5 × 15 B

= 75 B. | Day

Write calls to the Typeahead = 15B. | Day.

# of Reads = 75B

# of Writes = 15B.

→ R + W Heavy.

QPS.

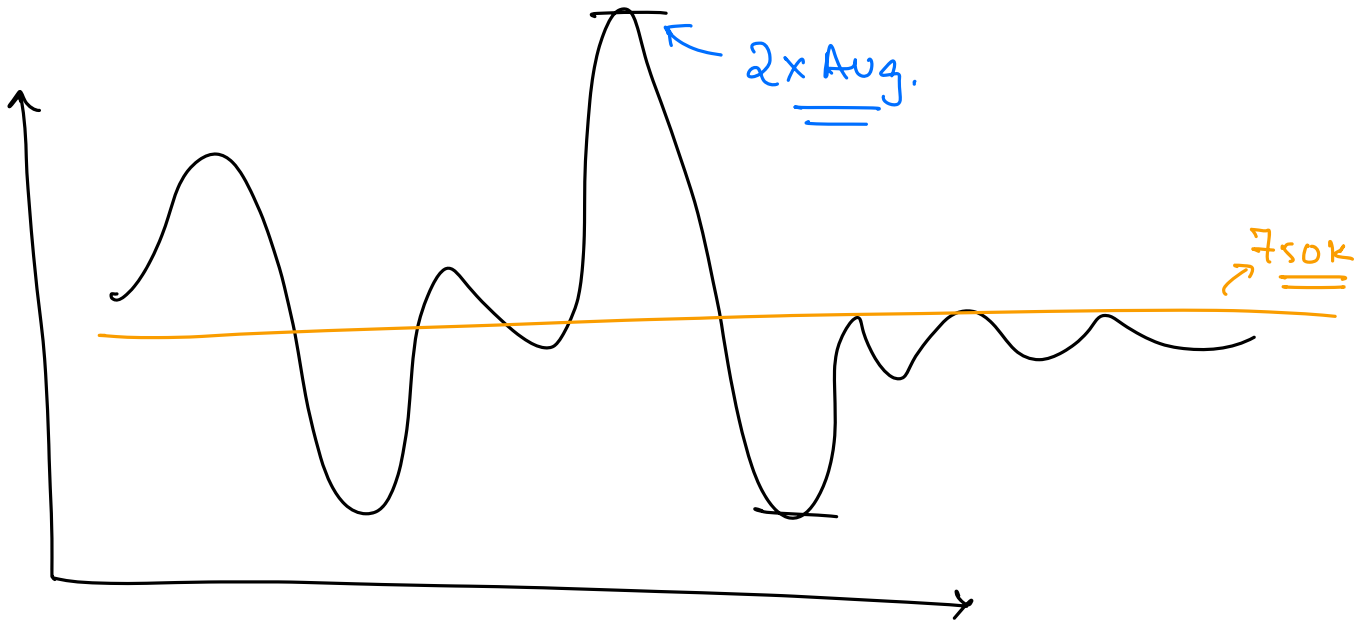
↳ Queries per sec.

Read qps = $\frac{75 \times 10^9}{24 \times 60 \times 60}$ = $\frac{75 \times 10^9}{86400}$ ≈ $\frac{75 \times 10^9}{100000}$

= 75×10^4

$= 750000 = 750K$

Writes = 150K.



Peak read qps = 2x Avg read qps

= 2x 750K

= 1.5M

Peak write qps = 2x Avg write qps.

= 300K.

Storage Calculations.

↳ 15 B / Day. = # of writes.

- Update frequency
- New term. → Storage

10% of new searches = 1.5 B.

query word | freq | Metadata ⇒ 100 Bytes

- query word → 30B
- freq → 8B
- Metadata → 60B

1 Day = $1.5 \times 10^9 \times 100$ Bytes.

= 150 GB.

10 Yrs = $150 \times 365 \times 10$

≈ 400

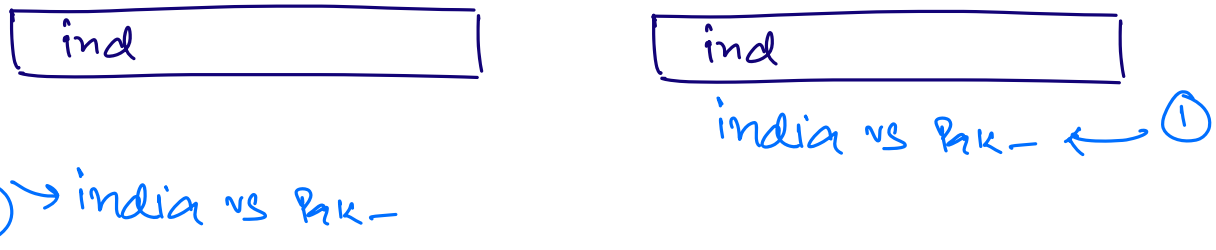
= 60×10^9 GB.

= 600 TB.

⇒ Sharding is required.

3. Tradeoffs.

⇒ High availability >>>> High Consistency.



⇒ Highly available but eventually consistent.

⇒ Ultra low latency.

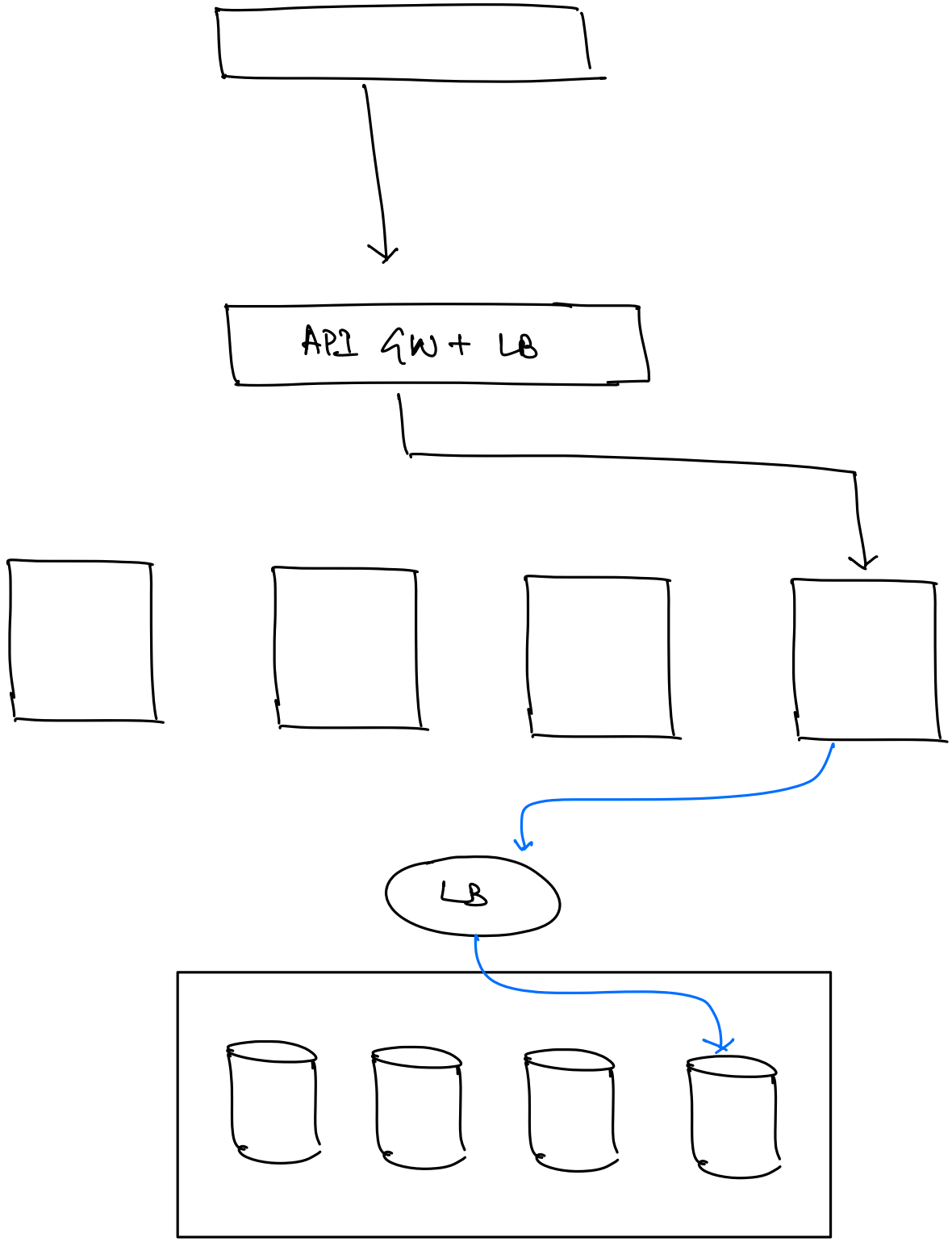
4. Design Goals & API's.

API's.

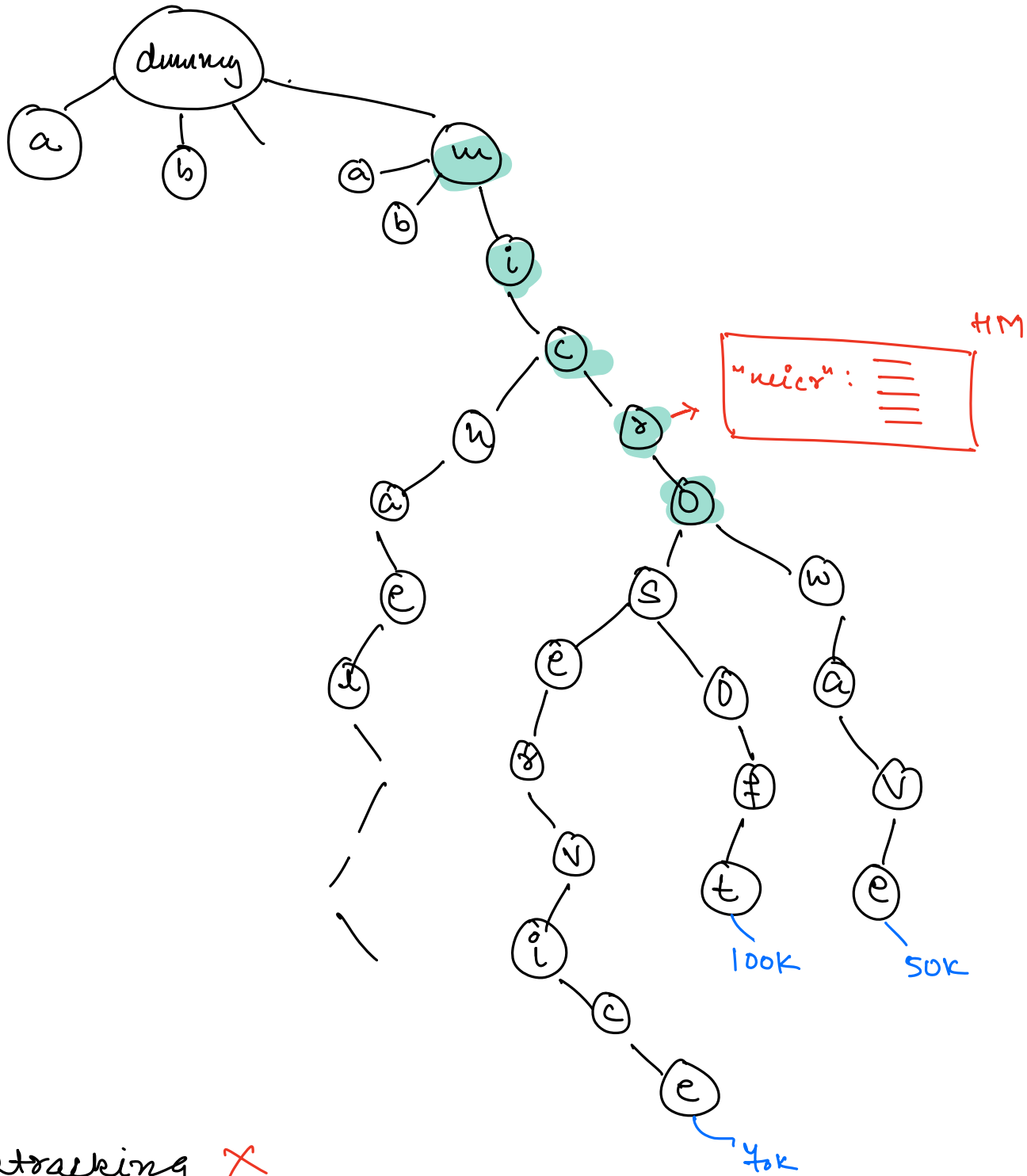
→ getSuggestions(query_prefix, limit = 5)

→ updatefreq(search_query)

Insert Update.


API GW + LB
LB

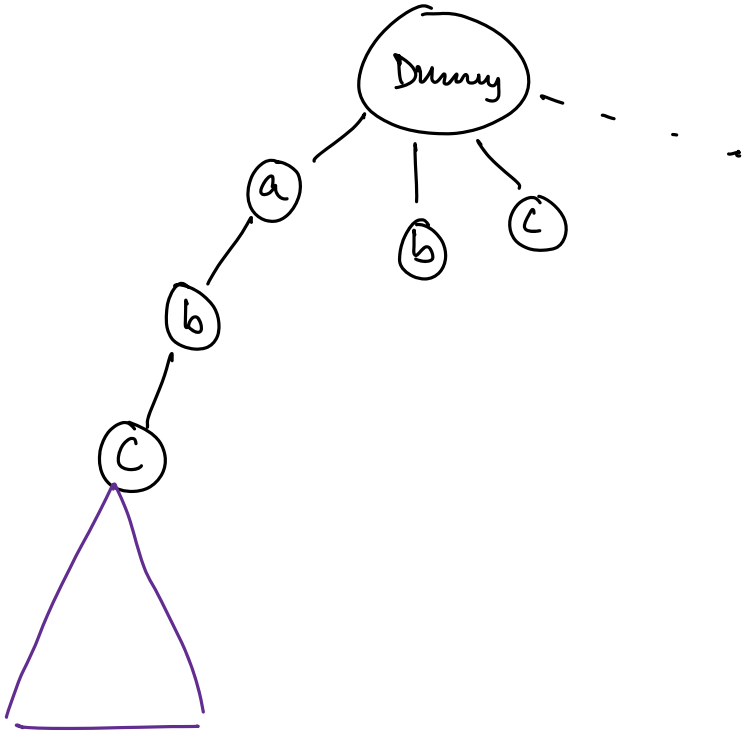
Trie



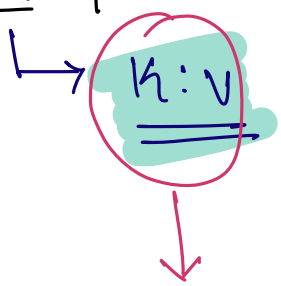
⇒ Backtracking. ✗

⇒ SHARD.

↳ Very complex to shard a Trie.



⇒ Hashmap.



Support Sharding automatically.